

Topic 8: Trees and Binary Trees

8.1 Tree Terminology

- Tree: hierarchical data structure with a root node and sub-trees of children
- Root: top node with no parent
- Leaf: node with no children
- Height of tree: longest path from root to a leaf (in edges)
- Depth of node: number of edges from root to that node
- Degree of node: number of children
- Binary tree: every node has at most 2 children (left and right)

8.2 Binary Tree Node

- Struct with data, left child pointer, right child pointer
- Left and right pointers are nullptr for leaf nodes

```
struct TreeNode {  
    int      data;  
    TreeNode* left;  
    TreeNode* right;  
    TreeNode(int d) : data(d), left(nullptr), right(nullptr) {}  
};
```

8.3 Tree Traversals

- Inorder (Left, Root, Right): produces sorted sequence for a BST
- Preorder (Root, Left, Right): copies a tree; used in expression trees
- Postorder (Left, Right, Root): deletes a tree; evaluates expression trees
- Level-order (BFS): visit level by level using a queue

```
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void preorder(TreeNode* root) {
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

8.4 Properties of Binary Trees

- Full binary tree: every node has 0 or 2 children
 - Complete binary tree: all levels filled except possibly the last, which is filled left to right
 - Perfect binary tree: all internal nodes have 2 children, all leaves at the same depth
 - A perfect binary tree of height h has $2^{h+1} - 1$ nodes
 - Skewed tree (worst case): degenerates to a linked list -- $O(n)$ operations instead of $O(\log n)$
-